

System V IPC

Review of IPC

▪ Inter Process Communication:

- Pipe
- Named Pipe (FIFO)
- Message Queue



Message Passing Model

- Shared Memory

Shared Memory Model

System V IPC interface

■ System V IPC provides

- Shared memory, Message Queue, Semaphores, ..
- APIs: `*get`, `*snd`, `*rcv`, `*ctl` (e.g. `msgget`, `msgsnd`, etc.)

■ Key and Identifiers

- Identifier: system-wide identifier for an **IPC object**
- Key: identify user (process) of IPC, **similar to file name**
- It can be hard coded or obtained by `ftok` function

■ Permission

- XYZ form, similar to file permission
- X : owner user, Y:owner group, Z: other users
- 100_2 : read (-r) 010_2 : write (-w) 001_2 : Not Used

System V IPC interface (cont.)

■ `key_t` `ftok`(`const char*` `path`, `int` `proj_id`)

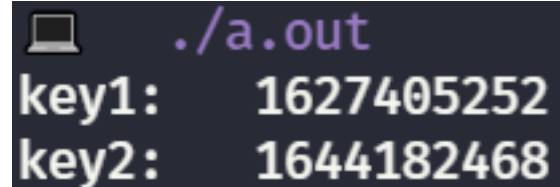
- To create deterministic key with string
- **path**: any valid file path
- **proj_id**: some integer must be non-zero (only lower 8 bits are used)

```
#include <stdio.h>
#include <sys/ipc.h>

int main() {
    key_t k1 = ftok(".", 'a');
    printf("key1:\t%d\n", k1);
    key_t k2 = ftok(".", 'a' + 1);
    printf("key2:\t%d\n", k2);
}
```

■ `int*` `get`(`key_t` `key`, `int` `flag`)

- return IPC object identifier
- **IPC_CREAT** (Common flag)
 - Create object if there is no object created by given key
- **IPC_EXCL** (Common flag)
 - combined with **IPC_CREAT** (**IPC_CREAT** | **IPC_EXCL**)
 - Return error if there is object created by given key



```
./a.out
key1: 1627405252
key2: 1644182468
```

System V IPC interface (cont.)

- `int* ~ctl(int ~id, int cmd, struct ~id_ds *buf);`
 - IPC control operations
 - **IPC_RMID** (Common cmd)
 - To remove object with given *id*,
 - Wake up waiting reader/writer processes with error returns
 - e.g., `msgctl(0x1234, IPC_RMID, NULL)`
 - **IPC_STAT** (Common cmd)
 - Get info of the kernel data structure for the object with given *id*
 - Return object specific data structure (e.g., *msqid_ds*, *shmid_ds*)
 - e.g., `msgctl(0x1234, IPC_STAT, ptr)`
 - **IPC_SET** (Common cmd)
 - Set fields of the kernel data structure for the object with given *id*
 - e.g., `msgctl(0x1234, IPC_SET, ptr)`

System V IPC interface (cont.)

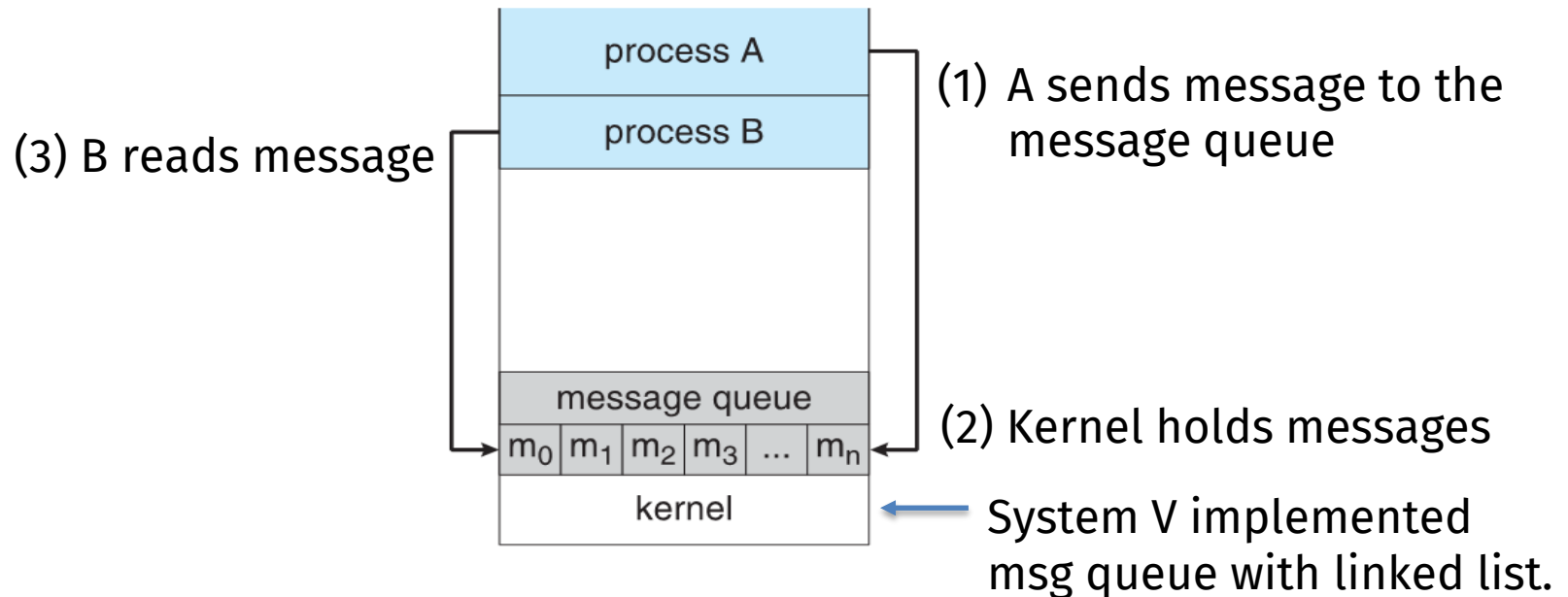
▪ `ipc_perm` (common data structure)

- Data structure designed to handle permission
- Used in the kernel data structure for IPC object
- Consists of user and group id, mode bit, and sequence number

```
// defined <sys/ipc.h>
struct ipc_perm {
    key_t      key;          /* user specified msg/sem/shm key */
    uid_t      cuid;         /* creator user id */
    gid_t      cgid;         /* creator group id */
    uid_t      uid;          /* user id */
    gid_t      gid;          /* group id */
    mode_t      mode;         /* r/w permission */
    unsigned short seq;      /* sequence # */
};
```

Message Queue

- Mail-box like communication method
- Typical message passing model
 - Pros : easy to use
 - Cons : overhead if data is large (copies in/out of kernel)



Message Queue

- Defined in `<sys/msg.h>`

```
struct msqid_ds {  
    struct ipc_perm msg_perm;  
    as a list struct msg *msg_first;  
    struct msg *msg_last;  
    resource unsigned long msg_cbytes;  
    unsigned long msg_qnum;  
    unsigned long msg_qbytes;  
    access pid_t msg_lspid;  
    pid_t msg_lrpid;  
    time_t msg_stime;  
    time_t msg_rtime;  
    time_t msg_ctime;  
};
```

/* permission */
/* first message */
/* last message */
/* number of bytes in use */
/* number of msgs */
/* max # of bytes */
/* pid of last sender() */
/* pid of last receiver() */
/* time of last send() */
/* time of last receive() */
/* time of last msgctl() */

Message Queue

▪ You have to implement msgbuf

- The first attribute of msgbuf is msgtype
- Sender and receiver should share the msgbuf implementation

```
struct msgbuf {  
    long msgtype;  
    // ...  
};
```

Must have the first attribute as msgtype
msgtype must be > 0 (POSITIVE)

```
struct msgbuf_text {  
    long msgtype;           1  
    char text[256];  
};
```

```
struct msgbuf_nums {  
    long msgtype;           2  
    int numbers[8];  
};
```

Message Queue

- `int msgget(key_t key, int msgflg);`

```
int mq_id = msgget (0x1234, IPC_CREAT | 0600); // RW of perm = 0600
```

```
struct msgbuf buf = { 1, /* initialize data */ };
```

- `int msgsnd (int msqid, const void* msgp, size_t msgsz, int msgflg);`

```
// msgflg default = 0 or IPC_NOWAIT(non-blocking)
```

```
msgsnd(mq_id, (void *)(&buf), sizeof(struct msgbuf), 0);
```

- `int msgrcv(int msqid, void* msgp, size_t msgsz, long msgtyp, int msgflg);`

```
// type: 0 == all msg , 0 > especially type , 0 < smallest absolute value type
```

```
msgsnd(mq_id, (void *)(&buf), sizeof(struct msgbuf), 1, 0);
```

Message Queue

- `int msgctl(int msqid, int cmd, struct msqid_ds *buf);`

e.g., `msgctl(mq_id, IPC_RMID, NULL); // remove msgq`

- See common command slide for **cmd**
 - **IPC_RMID** (Common cmd)
 - **IPC_STAT** (Common cmd)
 - **IPC_SET** (Common cmd)

Message Queue Example

<Sender>

```
int main(void) {
    key_t k = ftok(".", 'a');           //get key
    int qid = msgget(k, IPC_CREAT | 0660); //get msg queue object
    if (qid == -1) {
        perror("msgget error : ");
        exit(0);
    }
    struct msgbuf *msg = malloc(sizeof(struct msgbuf));
    msg->msgtype = 1;                   // set msgtype 1
    strcpy(msg->text, "Hello world?\n"); // fill up message

    /* send message */
    if(msgsnd(qid, (void *)msg, sizeof(struct msgbuf), 0) == -1)
    {
        perror("msgsnd error : ");
        exit(0);
    }
    free(msg);
}
```

```
struct msgbuf{
    long msgtype;
    char text[256];
};
```

Message Queue Example

<Receiver>

```
int main(void) {
    key_t k = ftok(".", 'a');           //get key
    int qid = msgget(k, IPC_EXCL | 0660); //get msg queue object
    if (qid == -1) {
        perror("msgget error : ");
        exit(0);
    }
    struct msgbuf *msg = malloc(sizeof(struct msgbuf));

    /* receive message */
    if(msgrcv(qid, (void *)msg, sizeof(struct msgbuf), 1, 0) == -1) {
        perror("msgsnd error : ");
        exit(0);
    }
    printf("%s", msg->text);
    free(msg);
}
```

```
struct msgbuf{
    long msgtype;
    char text[256];
};
```

Shared Memory Model

- **Mapping segment of memory that will be shared by more than one process**
- **Fastest form of IPC**
 - There is no intermediation!
- **Shared memory can be written to and read from by any number of processes**
- **Pros: Fast**
- **Cons: Synchronization**
- **Alternative:**
 - POSIX (named): `shm_open` -> `mmap` -> `munmap` -> `shm_unlink`
 - Linux (anonymous): `memfd_create` -> `mmap` -> fd transfer*
(`pidfd_getfd/socket`) -> `munmap` -> `close`

Shared Memory



<process 1 VM>

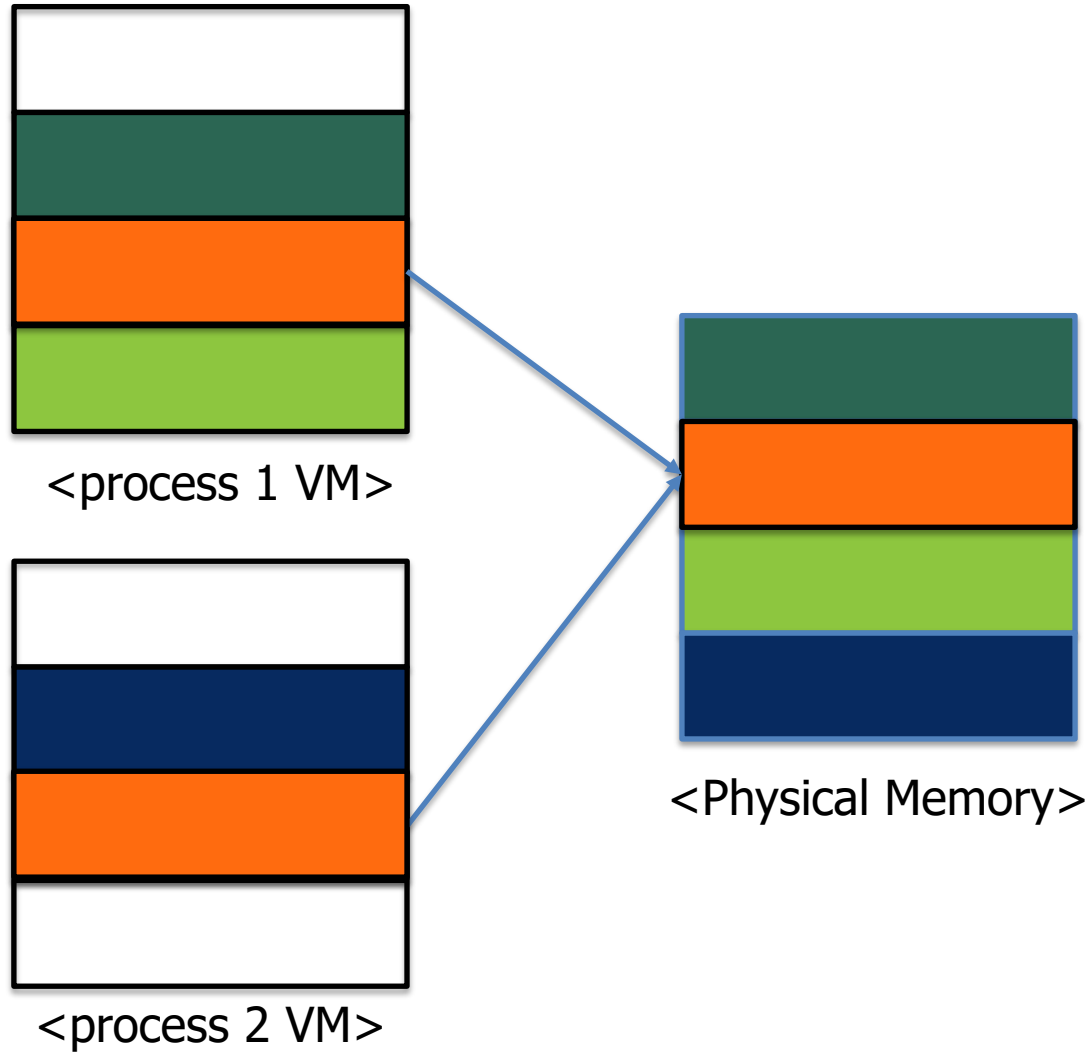


<process 2 VM>



<Physical Memory>

Shared Memory



Shared Memory

- `int shmget (key_t key, size_t size, int shmflag);`
 - `int id = shmget(0x1234, sizeof(int) * 100, IPC_CREAT);`
 - Use flags start with SHM_HUGETLB for reducing memory latency
- `void* shmat(int shmid, const void* shmaddr, int shmflag);`
 - shared memory attach
 - `void *ptr = shmat (id, NULL, 0);`
 - `shmaddr`: specify addr to attach
 - system chooses suitable address when NULL
 - `shmflag`: set 0 for READ WRITE permission
or if you want set readonly, set SHM_RDONLY

Shared Memory

▪ `int shmdt(int shmid, const void* shmaddr);`

- shared memory detach
- `int error = shmdt(id, shmaddr); // return 0 on success`
 - `shmaddr`: shared memory segment to detach
address should be returned address from `shmat()`

▪ `int shmctl(int shmid, int cmd, struct shmid_ds *buf);`

- shared memory control
- `shmctl(id, IPC_RMID, NULL); // remove shm specified by id`
- See common commands: `IPC_RMID`, `IPC_STAT`, `IPC_SET`

Shared Memory

```
// defined <sys/shm.h>
```

```
struct shmid_ds {
    struct ipc_perm    shm_perm;        /* operation perms */
    int                shm_segsz;       /* size of segment (bytes) */
    time_t             shm_atime;       /* last attach time */
    time_t             shm_dtime;       /* last detach time */
    time_t             shm_ctime;       /* last change time */
    pid_t              shm_cpid;        /* pid of creator */
    pid_t              shm_lpid;        /* pid of last operator */
    unsigned short      shm_nattch;     /* no. of current attaches */

    /*----- unused -----*/
    unsigned short      shm_unused;
    void*               shm_unused2;
    void*               shm_unused3;
};
```

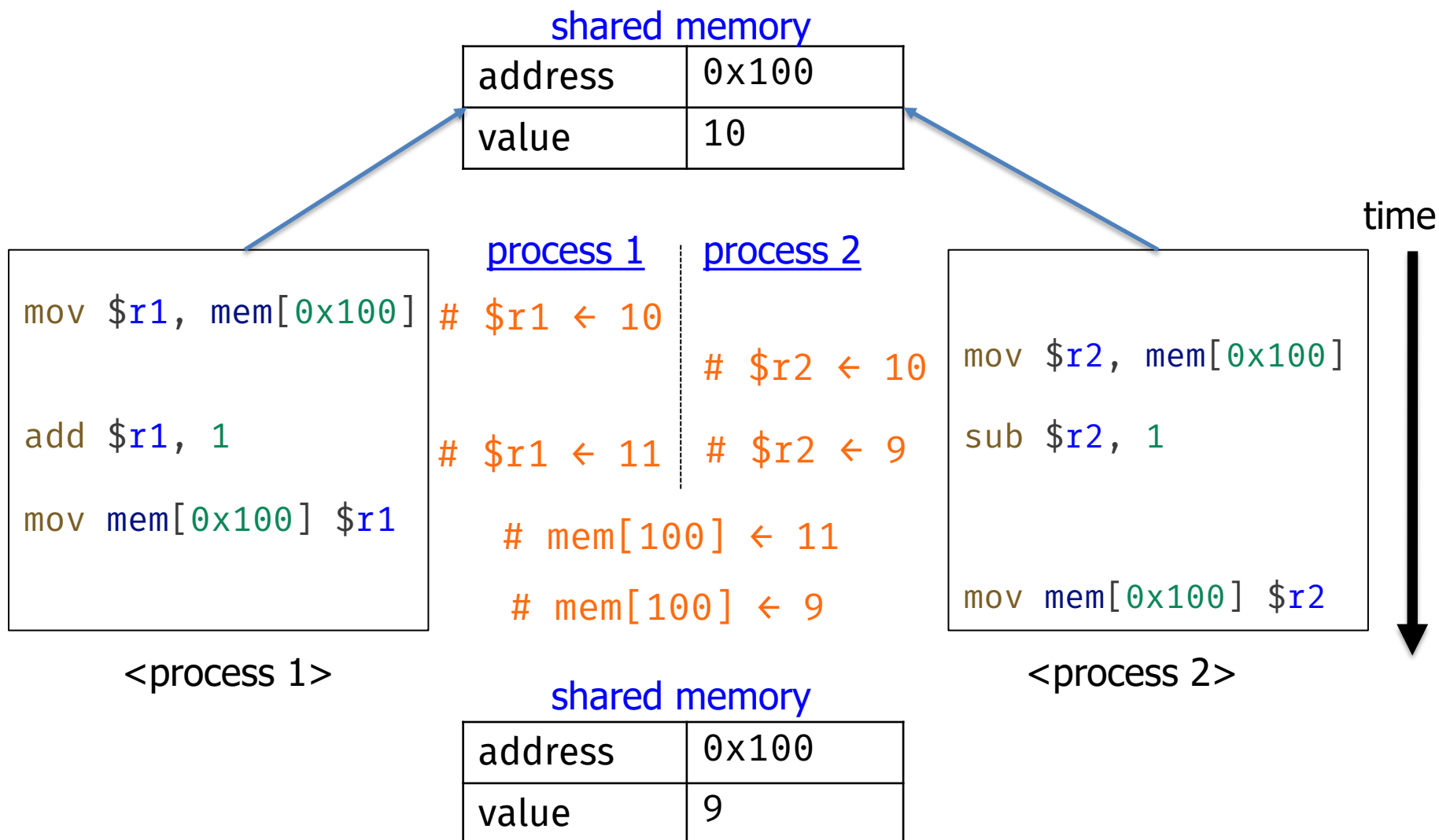
Shared Memory

```
int main(void) {  
    key_t k = ftok(".", 'b');  
    int shm_id = shmget(k, sizeof(int), IPC_CREAT | 0x1FF);  
    if (shm_id == -1) {  
        perror("shmget");  
        exit(1);  
    }  
  
    int* shm_addr = shmat(shm_id, NULL, 0);  
    *shm_addr = 1;  
    printf("value before fork: %d\n", *shm_addr);  
  
    switch (fork()) {  
        case -1:  
            perror("fork");  
            exit(2);  
        case 0:  
            *shm_addr += 1;  
            printf("value in child: %d\n", *shm_addr);  
            _exit(0);  
        default:  
            wait(NULL);  
            printf("value in parent: %d\n", *shm_addr);  
            shmctl(shm_id, IPC_RMID, NULL);  
    }  
  
    return 0;  
}
```

access permission
0x1FF = 0666

value before fork: 1
value in child: 2
value in parent: 2

Race Condition



Shared Memory

- Parent subtract 1 a million times while
- Child add 1 a million times
- Is the value in shm 1?

```
int main(void) {
    key_t k = ftok(".", 'b');
    int shm_id = shmget(k, sizeof(int), IPC_CREAT | 0x1FF);
    if (shm_id == -1) {
        perror("shmget");
        exit(1);
    }

    int* shm_addr = shmat(shm_id, NULL, 0);
    *shm_addr = 1;
    printf("value before fork: %d\n", *shm_addr);

    switch (fork()) {
        case -1:
            perror("fork");
            exit(2);
        case 0:
            for (int i = 0; i < 1000000; i++) *shm_addr += 1;
            _exit(0);
        default:
            for (int i = 0; i < 1000000; i++) *shm_addr -= 1;
    }
    printf("value after operation: %d\n", *shm_addr);
    shmctl(shm_id, IPC_RMID, NULL);
    return 0;
}
```

```
for i in {1..10};do ./a.out;done
value before fork: 1
value after operation: -335642
value before fork: 1
value after operation: -999999
value before fork: 1
value after operation: 402346
value before fork: 1
value after operation: -914993
value before fork: 1
value after operation: -630750
value before fork: 1
value after operation: -207743
value before fork: 1
value after operation: -813067
value before fork: 1
value after operation: -576624
value before fork: 1
value after operation: -325967
value before fork: 1
value after operation: -465761
```

Shared Memory

- **Add wait(NULL) code in the parent process**
- **The result is now 1 as desired**
- **For any process? : Semaphore, Lock ...**

```
switch (fork()) {
    case -1:
        perror("fork");
        exit(2);
    case 0:
        for (int i = 0; i < 10000000; i++)
            *shm_addr += 1;
        _exit(0);
    default:
        wait(NULL);
        for (int i = 0; i < 10000000; i++)
            *shm_addr -= 1;
}
```

[illegible]

Shell Commands

▪ **ipcs (-i, -m, -q, -a)**

- List IPC objects
- -i <id> : show details about given id
- -m : show shared memory
- -q : show message queue
- -a : show all

▪ **ipcrm (-M, -m, -Q, -q, -a)**

- Remove IPC objects
- -m<id> : remove shared memory given id (-M <key>)
- -q <id> : remove message queue given id (-Q <key>)
- -a : remove all IPC objects

Lab Exercise

- **Make chatting program using message queues**
 - Program gets user and receiver ID via STDIN when it starts
 - Users can receive messages while entering their message via fork
 - When the user receives a regular message from another user, print out the message
 - Once it reads the message, send back an ack message with read time to the other user
 - There should be no wait time between normal message and ack message (add **IPC_NOWAIT** flags to ***msgsnd/msgrcv***)
 - Guaranteed to have two users running the program at the same time (i.e., cases where there is only one user are ignored) with only one user typing at a given time.
 - Skeleton code on iCampus

Lab Exercise (cont.)

▪ Print Format

- User \$RCV_ID:\t\$USER_MSG // regular message
- User \$RCV_ID read message at \$READ_TIME // ack message
- Use localtime function + strftime (time.h) to match \$READ_TIME format

```
char buf[SIZE];
time_t t = time(NULL);
struct tm tm;
localtime(&t, &tm);
strftime(ack.timestamp, SIZE, "%Y-%m-%d %H:%M:%S", &tm);
```

Lab Exercise (cont.)

■ Example

```
User ID: 1
Receiver ID: 3
User 1: Hi!
User 3 read message at 2026-04-29 07:50:10
User 1: How are you?
User 3 read message at 2026-04-29 07:50:11
User 3: I'm fine!
User 3: Thank you!
User 3: And you?
User 1: quit
```

```
User ID: 3
Receiver ID: 1
User 1: Hi!
User 1: How are you?
User 3: I'm fine!
User 1 read message at 2026-04-29 07:50:14
User 3: Thank you!
User 1 read message at 2026-04-29 07:50:16
User 3: And you?
User 1 read message at 2026-04-29 07:50:17
User 3: quit
```

Exercise Submission

- **Submit your source code and Makefile in iCampus**

- Makefile should generate ./w9 executable
- Bundle *source code* and *Makefile* with tar command

```
$ tar caf [student_id]_w9.tar.zst week9
```

- We'll grade your submission with **make**
 - » If compilation fails, your points for this exercise will be zero